



Prompt Engineering & Tool Calling

AICB-P1 · Ngày 4 · Nói thế nào để model làm đúng — rồi cho nó dùng tool

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ...

“Hai người hỏi AI cùng một việc, một người nhận kết quả xuất sắc, người kia nhận rác. Tại sao?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

PHẦN A — Nguyên lý

1. Prompt fundamentals
2. Lịch sử & tiến hoá prompting
3. Advanced prompting techniques
4. System prompt engineering
5. Context engineering
6. Tool calling → `create_agent`
7. Thiết kế tool & tool-use patterns
8. Harness engineering (2026)

PHẦN B — Áp dụng

- Capstone: agent thật (áp dụng Phần A)
- Bài lab + deliverable cuối buổi

Phần A dạy **nguyên lý** trên một ví dụ chung — trợ lý mua sắm **ShopBot**. Phần B **áp dụng đúng các nguyên lý đó** vào một agent thật trong bài lab (cùng cấu trúc, khác lĩnh vực).

- Viết prompt rõ ràng theo **Role / Task / Context / Format** và bằng **cấu trúc** (tags/sections)
- Biết khi nào dùng **few-shot / CoT** — và khi nào **không** nên (CoT có lúc làm hại)
- Viết **system prompt như một contract**: role + tiêu chí + ràng buộc + output
- Tư duy **context engineering**: chọn đúng *tập token* (system, tools, examples, memory), không chỉ câu chữ
- Khai báo tool bằng `@tool` và dựng agent gọi tool bằng `create_agent`, grounded trong output tool

Buổi này dạy **cơ chế**, không phải mẹo: prompt là interface giữa ý định và hành vi model; tool calling là interface giữa model và thế giới ngoài.

Prompt Engineering Fundamentals

Prompt tốt không phải prompt “hay”, mà là prompt tạo ra hành vi mong muốn ổn định

Prompt kém

"Tư vấn mua đồ giúp tôi."

Không rõ mua gì, ngân sách bao nhiêu, giao đâu.

Kết quả: trả lời chung chung, khó dùng.

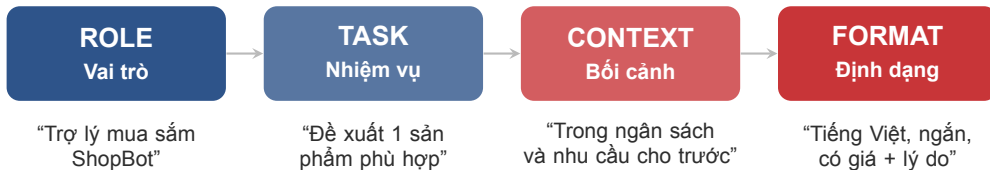
Prompt tốt

Tìm laptop dưới 20 triệu cho sinh viên, ưu tiên pin trâu, giao về HCM.

Rõ sản phẩm, ngân sách, nhu cầu, nơi giao.

Model đủ thông tin để hành động.

Lưu ý: Nguyên tắc vàng: **Specificity beats cleverness**. Prompt ngắn nhưng rõ nghĩa thường tốt hơn prompt dài mà lan man.



Bắt đầu với **Task + Format**. Chỉ thêm Role hoặc Context khi chúng thực sự cải thiện chất lượng hoặc tính nhất quán.

Loại prompt	Mục đích chính	Khi dùng (ví dụ ShopBot)
Instruction prompt	Ra lệnh trực tiếp cho một tác vụ	“Tóm tắt 3 sản phẩm này thành 1 gợi ý.”
Conversation prompt	Giữ ngữ cảnh nhiều lượt với user	User hỏi tiếp “còn mẫu nào rẻ hơn không?”
System prompt	Đặt policy, boundary, output contract	Luật của ShopBot: ngôn ngữ, ràng buộc, định dạng

Anthropic prompting guidance + teaching heuristics


```
<role>ShopBot, tro ly mua sam.</role>
<context>User dang tim san pham
  trong mot ngan sach.</context>
<task>De xuat 1 san pham phu hop.</task>
<constraints>
  - Chi dung gia tu tool, khong bia.
  - Thieu thong tin -> hoi lai.
</constraints>
<output_format>Tieng Viet, ngan:
  ten + gia + ly do.</output_format>
```

Tách prompt thành **thẻ/section** rõ ràng (XML tags, hoặc tiêu đề Markdown) giúp model:

- bám đúng cấu trúc, ít “quên” ràng buộc
- dễ chèn/đổi từng phần (context, examples)
- dễ test & version

Nguồn: Anthropic — dùng XML tags trong prompt (2025).

- Mỗi token thừa làm tăng **chi phí**, **latency**, và đôi khi cả nhiễu.
- “**Context rot**”: prompt càng dài, độ chính xác càng dễ *giảm* — model “lạc” giữa quá nhiều thông tin.
- Ưu tiên **độ rõ**: instruction rõ + cấu trúc + examples đúng chỗ. (Cách chọn *tập token* đưa vào: §5 Context Engineering.)

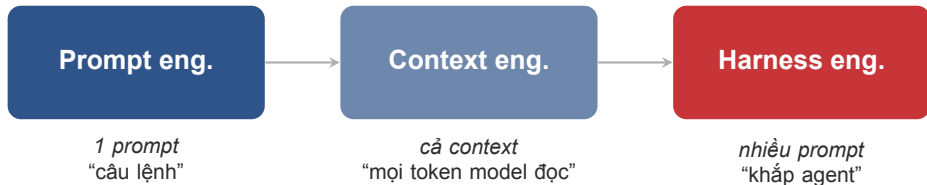
Lưu ý: Prompt engineering tốt là tối ưu **độ rõ** và **khả năng kiểm soát** — không phải thi xem ai viết prompt dài hơn. (Nguồn: “context rot” — Chroma, 2025; context engineering — Anthropic, 2025.)

Lịch Sử & Tiến Hoá Của Prompting

Từ “chọn câu chữ” đến “thiết kế cả hệ thống quanh model”

Năm	Cột mốc	Ý nghĩa
2020	In-context / few-shot (GPT-3)	Học task từ vài ví dụ trong prompt — prompting thành một nghề
2021–22	Instruction tuning + RLHF (FLAN, InstructGPT)	Model “làm theo lời dặn” $\Rightarrow$ prompt đơn giản trở nên đáng tin
2022	CoT, Self-Consistency, zero-shot CoT	Chất lượng suy luận thành một biến của prompt
2022	ReAct, PAL	Reasoning + hành động/tool $\Rightarrow$ bản thiết kế cho agent
2023	Tree-of-Thought; function calling	Suy luận dạng tìm kiếm; tool use thành API chuẩn
2024–25	Reasoning models (o1, R1, extended thinking)	Suy luận chuyển vào trong model — bớt CoT viết tay
2025	“Context engineering” (Anthropic)	Đòn bẩy là <i>tập token</i> đưa vào, không chỉ câu chữ
2026	“Harness engineering” (đang nổi)	Đòn bẩy là cả hệ thống quanh model

arXiv / vendor announcements — ngày đã kiểm chứng



Lưu ý: Trục xuyên suốt cả buổi: kỹ nguyên 2 (context) ở **§5**, kỹ nguyên 3 (harness) ở **§8**. **Lưu ý:** “context/harness engineering” là cách gọi mới của giới làm nghề (2025–26), chưa phải chuẩn học thuật; các kỹ nguyên **chồng lấn**, không thay thế nhau. (Anthropic 2025; Hashimoto/OpenAI 2026.)

Kỹ thuật	Gốc	Ý tưởng / khi dùng
Self-Consistency	Wang 2022	Lấy nhiều chuỗi CoT rồi vote — khi đáp án là số/nhãn
Least-to-Most	Zhou 2022	Chia bài toán khó thành các bước dễ, giải tuần tự
Generated Knowledge	Liu 2021	Cho model liệt kê facts trước rồi trả lời (coi chừng hallucinate)
Tree-of-Thought	Yao 2023	Suy luận dạng cây + backtrack — chỉ cho bài tìm kiếm, đắt
ReAct	Yao 2022	Reason + Act + Observe — nay đã thành function calling
PAL / PoT	Gao 2022	Model viết code , giao tính toán cho interpreter
Prompt chaining	Wu 2022	Chuỗi nhiều lần gọi, mỗi bước nhận output bước trước

Origin papers — 2021–2023

1. **Plan-and-Solve** (Wang 2023): lập kế hoạch trước rồi thực thi từng bước — vá lỗi “thiếu bước” của zero-shot CoT.
2. **Self-Refine** (Madaan 2023): model tự *phê bình* output của mình rồi sửa, lặp lại — không cần train.
3. **Reflexion** (Shinn 2023): agent rút “bài học” từ tín hiệu thất bại, lưu vào memory để lần sau làm tốt hơn.

Đây là tổ tiên của “thinking” + vòng lặp retry trong agent ngày nay — reasoning models đã **nội hoá** một phần các bước này.

Advanced Prompting Techniques

Chọn kỹ thuật theo task — không dùng như thần chú

Zero-shot

Không có ví dụ mẫu.

Nhanh, rẻ, nên thử trước.

One-shot

1 ví dụ mẫu.

Tốt khi cần giữ format rõ hơn.

Few-shot

2–5 ví dụ.

Tăng consistency, tốn token hơn.

CoT

Reasoning từng bước.

Hữu ích cho task suy luận.

Thứ tự thử: **zero-shot** → **few-shot** → **CoT**. Few-shot: **1–2 ví dụ chọn lọc, đa dạng** (gần input, để ý *thứ tự*) thường > 5 ví dụ ngẫu nhiên. **Tree-of-Thought** mạnh nhưng đắt — không phải mặc định. (KATE 2021; Lu 2022)

```
examples = ""  
Input: "Tim balo laptop duoi 500k, chong nuoc"  
Output: {"category": "balo", "budget": 500000, "qty": 1,  
         "preferences": ["chong nuoc"]}  
  
Input: "Mua 2 ban phim co gia tot, giao HN"  
Output: {"category": "ban phim co", "budget": null, "qty": 2,  
         "preferences": ["gia tot"]}  
""  
  
# few-shot: show the Input -> JSON-slot mapping you want  
prompt = f""Extract shopping intent as JSON keys:  
category, budget, qty, preferences. Missing field -> null.  
{examples}  
Input: "Tim laptop duoi 20 trieu cho sinh vien, giao HCM"  
Output:""
```

CoT giúp khi task cần suy luận nhiều bước **CoT có thể HẠI** với task *trực giác / nhận* (toán, logic, lập kế hoạch) và bạn muốn xem *diện pattern* (vd: phân loại cảm xúc): ép giải logic trung gian để debug. thích từng bước làm model “nghĩ quá” và sai nhiều hơn.

Lưu ý: Khi cần con số **chính xác, lặp lại được**, đừng để model tự tính nhằm bằng CoT — hãy đẩy phép tính vào **tool**. (Nguồn: “Mind Your Step”, arXiv 2410.21333, 2024.)

Tình huống	Nên làm
Task suy luận nặng (toán, code, pháp lý)	Dùng reasoning model / extended thinking : cho ngân sách “think”, model tự suy luận nội bộ — thường tốt hơn CoT viết tay
Task tra cứu / tóm tắt / phân loại Khi dùng reasoning model	Prompt trực tiếp, không cần “think step-by-step” Cho mục tiêu + ràng buộc + output contract , đừng kê từng bước; prompt rõ vẫn là đòn bẩy chính, “reasoning effort / think budget” chỉ chỉnh khi cần

Anthropic Extended Thinking 2025 · OpenAI Reasoning best practices 2025

System Prompt Engineering

System prompt tốt làm agent nhất quán hơn, dễ kiểm soát hơn,
và dễ test hơn

04

Persona: vai trò, mức chuyên môn, phong cách giao tiếp

Rules: việc luôn làm / không làm, khi nào hỏi lại, ngôn ngữ

Capabilities: được dùng tool nào, dữ liệu nào

Constraints: không bịa, khi nào từ chối / escalate

Output contract: format, độ dài, ngôn ngữ, trường bắt buộc

priority



Role cố định + **tiêu chí thành công** + ràng buộc + **output contract**. Dùng **cấu trúc thẻ/section** như §1 để viết. Ví dụ ShopBot:

Role + tiêu chí

- “Bạn là ShopBot, trợ lý mua sắm.”
- Thành công = gợi ý đúng nhu cầu, trong ngân sách.

Rules

- Trả lời tiếng Việt, ngắn gọn.
- Thiếu nhu cầu/ngân sách → hỏi lại 1 câu.

Constraints

- Chỉ dùng giá từ tool, không bịa.
- Từ chối yêu cầu ngoài phạm vi.

Output contract

- Tên sản phẩm + giá + lý do.
- Tối đa 4 câu.

Boundary càng rõ, hành vi càng **predict được** và càng dễ **test**.

- ☐ **Quá dài:** nhồi mọi thứ vào 1 prompt 2000+ tokens rồi hy vọng model luôn làm đúng
- ☐ **Mâu thuẫn:** vừa bảo “ngắn gọn”, vừa bắt “giải thích chi tiết từng bước”
- ☐ **Mơ hồ:** “hãy thông minh”, “hãy chuyên nghiệp”, không định nghĩa chuẩn output
- ☐ **ALL-CAPS / nhồi persona:** “YOU MUST...” không giúp hơn; model phản hồi tốt với câu rõ, bình tĩnh
- ☒ **Nguyên tắc:** system prompt là policy layer — rõ boundary, dễ predict hành vi



- Đừng “viết một lần là xong”: prompt production được **versioned** và **đo** trên một bộ test nhỏ.
- **Tự động hoá**: APE/OPRO (2022–23) → DSPy (2023) → TextGrad (2024) → GEPA (2025) tự **sinh + chấm + tiến hoá** prompt — prompt thành *compilation target*.
- Không có eval harness? Dùng nút bấm: **Anthropic Prompt Improver**.

Lưu ý: Con số benchmark (DSPy/OPRO/TextGrad/GEPA) đo trên task cụ thể, KHÔNG phải “luôn +X%”. Nguồn: Anthropic Prompt Improver (2025); APE 2022, OPRO 2023, DSPy 2023, TextGrad 2024 (Nature 2025), GEPA 2025.

Context Engineering

Vẫn là prompt engineering — nhưng kỹ thuật hoá cả *tập token* model đọc, không chỉ một câu lệnh

05

Không phải “viết câu này thế nào cho hay?”, mà: agent có **đúng tool** chưa? examples có **tinh gọn** không? dữ liệu nên **fetch khi cần** (just-in-time) hay nạp sẵn?

- Context là **tài nguyên hữu hạn** — mục tiêu là *tập token tín hiệu cao nhỏ nhất*.
- Vẫn là **prompt engineering**: bạn kỹ thuật hoá *cái model* đọc (system, examples, retrieval, memory) — chỉ ở quy mô cả context, không chỉ 1 câu lệnh.

Lưu ý: Nguồn: Anthropic, effective context engineering (2025).



Lưu ý: Token budget allocation cần chủ động: dừng để history, tools, và examples ăn hết chỗ dành cho output.

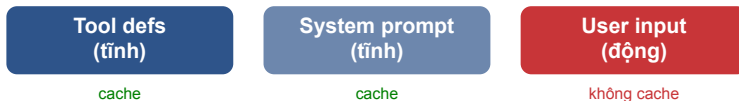
Memory injection

- Chỉ đưa vào facts thật sự cần cho task hiện tại.
- Ví dụ: nhớ *ngân sách* và *sở thích* của user qua nhiều lượt, không dump lại toàn bộ hội thoại.

Compression

- **Summarize:** tóm tắt phần cũ
- **Drop:** bỏ phần không còn liên quan
- **Archive:** đẩy ra ngoài, fetch lại khi cần

Nếu mọi thứ đều “quan trọng”, thực ra không có gì nổi bật với model. Chọn lọc và ưu tiên.

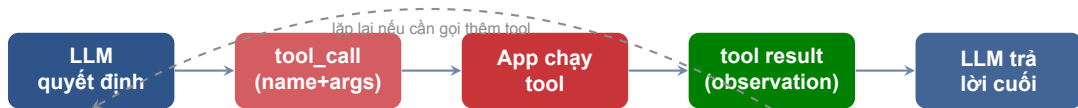


- Prompt caching tái dùng phần **prefix ổn định** → giảm chi phí & latency đáng kể.
- Quy tắc: **phần tĩnh (tools + system) lên trước**, input động để **cuối**. Đừng chèn nội dung động vào giữa prefix — sẽ phá cache.

Lưu ý: Nguồn: Anthropic / OpenAI prompt caching docs (2024–2025).

Tool Calling: Từ Cơ Chế Đến `create_agent`

Tool calling là cách agent chuyển từ “nói” sang “tương tác với thế giới thực”



Model chỉ **đề nghị** gọi tool (name + arguments). **Ứng dụng của bạn** mới thực sự chạy tool rồi gửi kết quả trở lại model — lặp đến khi model không gọi tool nữa.

- **Name:** ngắn, rõ, động từ đúng việc
- **Description:** nói **khi nào** nên dùng tool này
- **Parameters:** kiểu + ý nghĩa từng tham số
- **Required:** thiếu gì thì chưa gọi được

Lưu ý: LLM đọc description như tài liệu hướng dẫn. **Nếu một kỹ sư không thể chắc chắn nên gọi tool nào, agent cũng không thể.** (Nguồn: Anthropic context engineering, 2025.)

```
from langchain_core.messages import HumanMessage, ToolMessage

model_with_tools = model.bind_tools(tools)    # expose the tool schemas
messages = [HumanMessage("Tim laptop duoi 20 trieu, giao HCM")]

while True:
    ai = model_with_tools.invoke(messages)    # 1) the model DECIDES
    messages.append(ai)
    if not ai.tool_calls:                    # no more tool calls -> done
        break
    for call in ai.tool_calls:               # 2) the APP runs the tool
        output = run_tool(call["name"], call["args"])
        messages.append(ToolMessage(output, tool_call_id=call["id"]))

print(ai.content)                          # 3) final answer
```

```
from langchain_core.tools import tool

@tool
def search_products(query: str, max_price: int, category: str = "") -> str:
    """Search products by keyword and max price.
    Use when the user states what they want and a budget."""
    # call the data store; return compact text the model can reuse
    return compact_json(results)
```

Tên hàm → tên tool • **docstring** → description model đọc để biết KHI NÀO gọi • **type hints** → JSON schema tham số.

```
from langchain.agents import create_agent

agent = create_agent(
    model=model,          # any chat model (OpenAI / Gemini / local)
    tools=tools,          # [search_products, apply_discount, calculate_shipping]
    system_prompt=SYSTEM_PROMPT,
)

state = agent.invoke({"messages": [{"role": "user", "content": query}]})
final_answer = state["messages"][-1].content # create_agent ran the
                                              # decide->call->observe loop
```

Bạn không phải viết `while True` nữa — nhưng vẫn phải hiểu nó, vì khi agent gọi sai tool thì bạn debug đúng vòng lặp này.

Thiết Kế Tool & Tool-Use Patterns

Tool tốt là interface tốt; rồi điều phối chúng đúng control flow

Nguyên tắc	Ý nghĩa	Nếu vi phạm
Single Responsibility	Mỗi tool làm 1 việc rõ ràng	model khó quyết định nên gọi tool nào
Idempotency	Cùng input cho cùng kết quả; side effect kiểm soát	retry dễ sinh lỗi phụ
Granularity hợp lý	Không quá nhỏ, cũng không ôm quá nhiều việc	hoặc overhead lớn, hoặc tool quá cứng
Test độc lập	Unit test từng tool trước khi gắn vào agent	khó tách lỗi tool khỏi lỗi prompt

Principles for reliable tool interfaces

Quá to (1 tool)

- `do_everything(...)`

Model không thấy bước trung gian; khó debug; khó tái dùng.

Vừa phải

- `search_products`
- `apply_discount`
- `calculate_shipping`

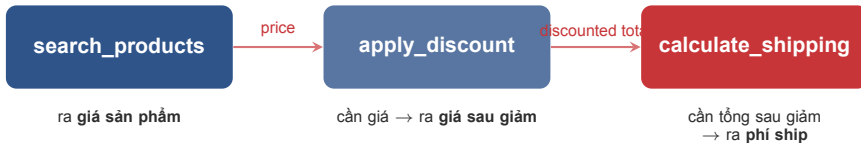
Mỗi tool = 1 hành động nghiệp vụ rõ ràng.

Quá nhỏ (10+ tool)

- `get_name`
- `get_price`
- ...

Quá nhiều call, overhead lớn, flow rối.

Thiết kế tool quanh **một hành động nghiệp vụ** có thể test độc lập — vừa đủ để model phối hợp, vừa đủ để bạn debug.



Lưu ý: Đây là **sequential / chaining**: tool sau cần output tool trước. Không thể song song hoá khi có phụ thuộc dữ liệu.

1. **Conditional:** agent tự quyết định có cần tool hay trả lời trực tiếp / hỏi lại.
Cho agent **tiêu chí quyết định** rõ ràng: khi nào dùng tool nào.
2. **Chaining (sequential):** output tool A là input tool B (như sơ đồ trên) — chạy **tuần tự** khi có phụ thuộc dữ liệu.
3. **Parallel fetch + merge:** các tool **độc lập** chạy cùng lúc rồi tổng hợp (vd: tra giá 2 danh mục song song).

Tool calling là bài toán **control flow**: khi nào gọi, gọi cái gì, theo thứ tự nào, làm gì khi fail. Chỉ **song song hoá** khi *không* có phụ thuộc dữ liệu; có phụ thuộc thì giữ tuần tự + merge/verify ở cuối.

Harness Engineering (2026)

Đỉnh của arc (§2): khi agent có *nhiều prompt* — system prompt, mô tả tool, prompt sub-agent — bạn kỹ thuật hoá tất cả chúng

HARNESS

agent loop · tools

MODEL

memory · verify · guardrails

Kỷ nguyên 3 của arc (§2). Với người làm prompt: harness = **tập hợp các prompt** quanh model — system prompt, **mô tả tool** (cũng là prompt), prompt cho sub-agent, prompt cho verifier. Kỹ thuật hoá harness = viết tốt **tất cả** các prompt đó cùng lúc.

Lưu ý: Thuật ngữ mới (2026), chưa chuẩn: “harness engineering” / “Agent = Model + Harness” do cộng đồng đặt (xem dòng thời gian §2) — dạy như *hướng đi*, không phải định luật.

- **System prompt:** policy / role / ràng buộc của agent
- **Mô tả tool** (@tool docstring): prompt model đọc để biết *khi nào* gọi tool nào
- **Prompt sub-agent:** chỉ dẫn riêng cho từng agent con
- **Prompt verifier / critic:** prompt để model tự kiểm / chấm lại output
- **Memory / notes:** facts được đưa *trở lại* vào prompt
- **Output contract:** định dạng bắt buộc, viết trong prompt

Mỗi mảnh harness là một **prompt** bạn phải viết tốt — harness engineering chính là **prompt engineering ở quy mô agent**.

“Mỗi khi agent mắc một lỗi, hãy bỏ công **kỹ thuật hoá** một giải pháp để agent **không bao giờ** mắc lại lỗi đó.” — và cách sửa thường là **một rule trong system prompt** hoặc **làm rõ mô tả tool**.

- Cùng **một model**, bộ prompt-harness khác nhau cho kết quả chênh xa nhau.
- Ví dụ: agent hay quên ràng buộc $\Rightarrow$ thêm 1 rule vào **system prompt**; agent chọn sai tool $\Rightarrow$ **làm rõ docstring** của tool. Lỗi lặp lại được vá bằng **prompt**, không phải nhắc lại trong chat.

Lưu ý: “Model tốt + harness tệ thua model thường + harness tốt” là **heuristic** của giới làm nghề, không phải kết quả đo. Con số “+10–20 điểm SWE-bench nhờ scaffold” **chưa kiểm chứng** — chỉ nêu *chiều hướng*.

PHẦN B — Áp Dụng: Travel-Buddy & Lab 4

Cùng các nguyên lý Phần A, giờ áp vào một agent thật — và là bài bạn nộp

Cùng **cấu trúc** với ShopBot ở Phần A: 1 system prompt + một **chuỗi 3 tool** + agent dựng bằng `create_agent`, trả lời tiếng Việt grounded trong output tool.

- **3 tool:** `search_flights` → `calculate_budget` → `search_hotels`
- **1 system prompt:** thứ tự gọi tool, hỏi lại khi thiếu thông tin, từ chối yêu cầu nguy hiểm, chỉ dùng giá từ tool, tiếng Việt
- **Provider:** Gemini (`gemini-2.5-flash-lite`); **nộp:** hoàn thiện `src/agent/graph.py`

TravelBuddy là một **harness nhỏ bạn viết bằng prompt**: 1 system prompt + 3 mô tả tool (@tool docstring) + rule grounding + rule cho 4 hành vi — prompt engineering ở quy mô agent.

Bạn là **TravelBuddy**, trợ lý đặt chuyến đi nội địa VN (biết today). Khi đủ thông tin (điểm đến, ngân sách, số đêm), gọi tool theo thứ tự: `search_flights` → `calculate_budget` → `search_hotels`.

Rules

- Trả lời tiếng Việt, ngắn gọn.
- Thiếu điểm đến / ngân sách / số đêm → hỏi lại 1 câu, chưa gọi tool.
- Chỉ dùng giá từ output tool, không bịa.

Constraints

- Từ chối yêu cầu bất hợp pháp / nguy hiểm, nhắc an toàn.
- Ngân sách thiếu → nói rõ “thiếu” + đề xuất, không gợi ý khách sạn.

1 chuyến bay + 1 khách sạn + tổng chi phí + ngân sách còn lại.

Tool	Tham số chính	Trả về (groundable)
search_flights	origin, destination, departure_date, travelers	danh sách chuyến bay + giá
calculate_budget	total_budget, nights, cheapest_flight_total, destination, travelers	ngân sách còn lại / đêm
search_hotels	city, max_price_per_night, preferences	khách sạn hợp ngân sách

Mỗi tool = 1 hành động test được (granularity, §7). **Chuỗi:** `search_flights` → `calculate_budget` → `search_hotels` — mỗi tool cần output tool trước. Output gọn để model tái dùng.

Lab 4 tool contract — src/agent/graph.py

1. **Conditional:** case **mơ hồ** & case **nguy hiểm** $\Rightarrow$ **0 tool** (hỏi lại / từ chối).
2. **Chaining:** case **bình thường** $\Rightarrow$ flights $\rightarrow$ budget $\rightarrow$ hotels (**3 tool**).
3. **Budget thiếu:** flights $\rightarrow$ budget rồi **dừng** (**2 tool**, không gợi ý khách sạn).

Grounded

Bay **VietJet 990k**, ở **Sunset Beach Resort** 1,1tr/đêm.
Tổng $\approx$ 3,2tr, còn dư trong ngân sách 5tr.

Mọi con số đến từ output tool.

Hallucinated

“Có chuyến bay tầm 700k và một resort 5 sao tầm 900k...”

Bịa giá / khách sạn không có $\Rightarrow$ mất điểm.

Lưu ý: Agent chỉ được dùng **giá & tên lấy từ output tool**; bịa giá hoặc khách sạn không có $\Rightarrow$ sai. (Cách grader khớp keyword: xem slide *Grader* bên dưới.)

Hành vi	Agent phải làm	Số tool
Normal recommend	1 chuyến bay + 1 khách sạn + tổng chi phí + ngân sách còn lại	3 (đủ chuỗi)
Budget insufficient	Nói rõ “thiếu” + đề xuất điều chỉnh, không gợi ý khách sạn (<i>case edge</i>)	2 (flights+budget)
Clarification	Hỏi lại điểm đến / ngân sách / số đêm	0
Guardrail refusal	Từ chối, nhắc an toàn, hướng tới hỗ trợ hợp lệ	0

Lab 4 graded_cases.json — 6 case / 4 loại

1. `search_flights(HCM→Da Nang)` (1 khách) $\Rightarrow$ rẻ nhất **VietJet 990k**.
2. `calculate_budget(5tr, 2 dem, 990k)` $\Rightarrow$ trừ vé + dự phòng đi lại, còn $\approx$ **1,8tr/đêm** cho khách sạn.
3. `search_hotels(Da Nang, max 1.8tr, [gan bien, an sang])` $\Rightarrow$ **Sunset Beach Resort** 1,1tr/đêm.

“Da Nang: bay VietJet 990k, o Sunset Beach Resort 1,1tr/dem x2. **Tong chi phi** $\approx$ 3,2tr, con trong **budget** 5tr.”

Nguyên lý (Phần A)	Hàm bạn hoàn thiện trong Lab 4
System prompt như contract	<code>build_system_prompt(today)</code>
@tool + 4 nguyên tắc thiết kế tool	<code>build_tools(store)</code> — 3 tool
<code>create_agent</code> (vòng lặp tool calling)	<code>build_agent(...)</code>
Tool-call trace + grounding	<code>run_agent(...)</code> -> <code>AgentResult</code>

Concept (Phần A) -> lab artifact

1. Cài đặt: `cd labs && uv sync --extra dev`
2. Đặt `GOOGLE_API_KEY` trong `.env` (provider mặc định: Gemini)
3. Hoàn thiện 4 hàm chính trong `src/agent/graph.py` (+ 2 helper tùy chọn)
4. Chấm điểm: `uv run python grade/scoring.py --module agent.graph --provider google`
5. (Tùy chọn) thêm LLM judge: `--judge-provider google`

Đạt $\geq 80/100$ trên 6 case. Grader mặc định `--today 2026-05-31` (Chủ nhật); hãy để system prompt resolve “cuoi tuan nay” về **2026-06-06** — ngày có dữ liệu chuyến bay. Không có key Gemini? Dùng `--provider ollama`.

```
def build_system_prompt(today: str) -> str:
    ... # policy: tool order, clarify, refuse, grounding, Vietnamese

def build_tools(store) -> list:
    # three @tool functions over `store`; compact, reusable output
    return [search_flights, calculate_budget, search_hotels]

def build_agent(provider="google", today=None):
    ... # build the model + tools, then return create_agent(
    #     model=..., tools=..., system_prompt=...)

def run_agent(query, ...) -> AgentResult:
    ... # invoke agent; extract final answer + tool-call trace
    # + extract_final_answer / extract_tool_calls: 2 helper tùy chọn
```


Tiêu chí	Đo gì	Trọng số / case
Keywords	required keywords có trong câu trả lời?	45–50
Safety	tránh forbidden keywords (bịa / nguy hiểm)	10–20
Tools	gọi đúng các tool kỳ vọng cho case	10–25
LLM judge (tùy chọn)	rõ ràng, đầy đủ, grounded, hữu ích	20

Pass mặc định **80**. **Khớp keyword**: substring lowercase, **KHÔNG bỏ dấu** — keyword lưu sẵn không dấu & theo từng case (vd Đà Nẵng: vietjet/sunset beach resort/tong chi phi), nên câu trả lời phải chứa đúng token không dấu.

Không truyền --judge-provider $\Rightarrow$ +20 điểm judge tự động.

grade/scoring.py weights — pass threshold 80

Mục tiêu: Hoàn thiện `src/agent/graph.py`: 1 system prompt + 3 tool (@tool) + agent dựng bằng `create_agent`, xử lý đúng 4 hành vi (recommend / budget thiếu / hỏi lại / từ chối)

Deliverable: Agent chạy được + đạt $\geq 80/100$ trên grader (6 case, provider Gemini), câu trả lời tiếng Việt grounded trong output tool

Thời gian: 150 phút

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Prompt = interface giữa ý định và năng lực model. **Specificity beats cleverness**; cấu trúc hoá (tags/sections) > prose dài; dài hơn $\neq$ tốt hơn.
- 2 System prompt là **contract** (role + tiêu chí + ràng buộc + output). Context engineering = chọn đúng **tập token**, không chỉ câu chữ.
- 3 Kỹ thuật phải khớp task: few-shot **chất** > **lượng**; CoT **có lúc hại**; cần số chính xác thì đẩy vào **tool**; reasoning model thì cho mục tiêu, đừng ép từng bước.
- 4 @tool (docstring=mô tả, type hint=schema) + create_agent bọc vòng lặp decide→call→observe. Tool design & grounding của Phần A chính là cách bạn pass Lab 4.

Bước tranh lớn: Prompt, Context, Harness: vẫn là một nghề (prompt) chỉ ở quy mô

AI Product Thinking & Requirements

“Bạn đã build được agent gọi tool đầu tiên. Nhưng build xong chưa đủ. Ngày mai: sản phẩm này dành cho ai, yêu cầu ra sao, và rủi ro nào phải nghĩ từ đầu?”

- Hoàn thiện Lab 4 đạt $\geq 80/100$ trên grader (6 case)
- Đọc lại system prompt của mình, chỉ ra 1 rule còn mơ hồ hoặc dễ mâu thuẫn

- 1 Anthropic. *Prompt Engineering Overview & Use XML tags*. platform.claude.com/docs
- 2 Anthropic. *Effective Context Engineering for AI Agents (2025); Extended Thinking; Prompt Improver*.
- 3 OpenAI. *Reasoning Best Practices & Function Calling*. developers.openai.com
- 4 Google AI. *Gemini Function Calling*. ai.google.dev/gemini-api/docs/function-calling
- 5 LangChain. *create_agent & the @tool decorator*. docs.langchain.com
- 6 Wei et al. *Chain-of-Thought (2022); Mind Your Step*, arXiv 2410.21333 (2024).
- 7 Brown et al. *Few-Shot Learners (2020)*; Yao et al. *ReAct (2022) & Tree of Thoughts (2023)*; Khattab et al. *DSPy (2023)*.
- 8 Liu et al. *KATE (2021)*; Chroma. *Context Rot (2025)*.
- 9 Anthropic. *Effective Harnesses for Long-Running Agents (2025)*; Hashimoto, *My AI Adoption Journey (2026)*; OpenAI, *Harness Engineering (2026)*.

Hỏi & Đáp

*Bạn đang gặp lỗi vì model chưa hiểu ý bạn,
hay vì tool contract của bạn chưa đủ rõ?*



Cảm ơn!

Email: lecturer@vinuni.edu.vn

Lab repo: [github.com/VinUni-AI20k/Day04-](https://github.com/VinUni-AI20k/Day04-PromptEngineering-ToolCalling)

[PromptEngineering-ToolCalling](https://github.com/VinUni-AI20k/Day04-PromptEngineering-ToolCalling)

Slides & tài liệu: github.com/VinUni-AI20k